

AI Agent 系统设计

从经典计算机工程到现代智能体架构 - 第一部分：前三章完整版

Working Draft v0.2 - 2026-06-29

前言

这份文档不是一份 Agent API 使用手册，也不是一次聊天记录整理。它试图从软件工程师和分布式系统架构师的角度，解释为什么现代 AI Agent 的架构越来越像经典计算机系统。

本文的基本观点是：LLM 是最近几年 AI 的核心突破，但当 LLM 被放进真实产品和真实工作流之后，很多问题又重新回到了计算机工程最熟悉的领域：如何管理状态、如何降低远程调用成本、如何做缓存和索引、如何保持服务无状态、如何在多个计算资源之间调度。

第一部分先完成前三章：第 1 章建立整体视角；第 2 章把 LLM 放回“计算引擎”的位置；第 3 章说明 Agent 为什么更像 Orchestrator，而不是简单的聊天机器人。

核心主线：LLM 是新的 Compute Engine，Agent 是围绕它进行上下文管理、工具调用、状态恢复和资源调度的 Orchestrator。

目录大纲

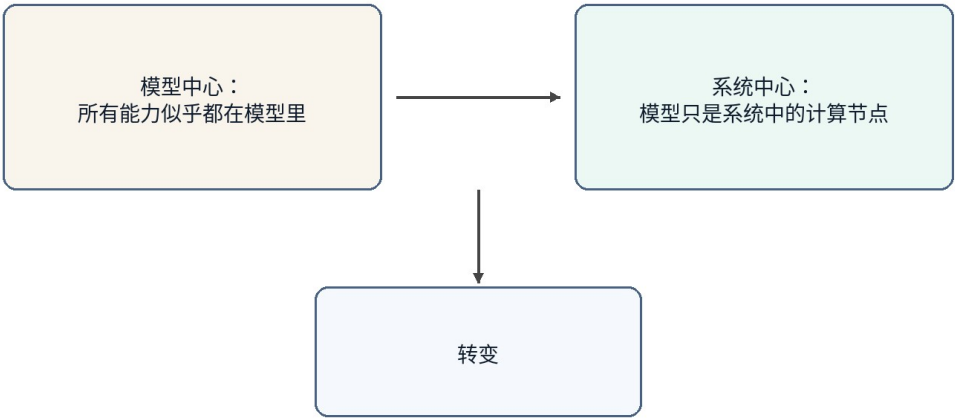
章节	标题	状态
第 1 章	为什么 AI Agent 让我想到了经典计算机工程	已完成
第 2 章	LLM：一种新的 Compute Engine	已完成
第 3 章	Agent：为什么它更像 Orchestrator	已完成
第 4 章	Memory、Tool 与 Planner 在 Agent 中的	后续
第 5 章	Compute / Storage Separation 在 AI 中的体现	后续
第 6 章	Stateless Agent 与微服务设计	后续
第 7 章	Context Engineering 与 Query Optimization	后续
第 8 章	AGENTS.md 与 Prompt Index	后续
第 9 章	RAG、Retrieval 与 Context Routing	后续
第 10 章	Token Reduction、Distillation 与 Tiered Compute	后续
第 11 章	Agent 生可靠性：等、待机与回放	后续
第 12 章	Multi-Agent、并发调度与多租户	后续

第 13 章	Agent 安全：Prompt Injection、Sandbox 与权限边界	后续
第 14 章	未来方向：Agent Operating System	后续

术语约定

术语	本文中的含义	工程类比
LLM	大言模型，推理算	Compute Engine
Agent	围绕 LLM 组织 Memory、Tool、Planner、Context 的系统层	Orchestrator / Runtime
Memory	期保存的用偏好、目背景和任状	Database / Cache
Context	本次求真正送入模型的工作集	Working Set / Buffer Pool
Tool	Agent 可调用的外部能力，例如文件、件、日、GitHub、Shell	RPC / API
Distillation	把大模型能力迁移到小模型或固定工作流	Precomputation / Tiered Compute
Sandbox	限制 Agent 工具、文件、网络 and 代行权限的隔离境	OS Process / Container
Idempotency	保证重试不会重复产生副作用的执行约束	Payment Idempotency / Exactly-once Boundary
Replay	复现 Agent 执行过程、工具调用和状态变化，用于调试、审计和对账	Event Log / Audit Trail

从模型中心到系统中心



图示为抽象架构，不代表某个产品的官方实现。

图 1

图 1：AI Agent 的讨论正在从模型中心转向系统中心

第 1 章 为什么 AI Agent 让我想到了经典计算机工程

本章定位：建立从模型能力到系统架构的观察角度。

1.1 本章问题

过去几年，AI 的公众叙事通常围绕模型展开：模型更大了，推理更强了，编码更好了，长上下文更长了。这当然是事实，但如果只盯着模型，很容易忽略另一个正在发生的变化：AI 产品正在从“一个模型回答问题”变成“一个系统完成任务”。

一旦从回答问题转向完成任务，问题就不再只是模型能力，而是系统能力。系统需要保存长期状态，需要访问外部工具，需要理解当前工作目录，需要决定哪些历史信息应该进入上下文，需要在成本、延迟和正确性之间做权衡。这些问题并不陌生，它们正是经典计算机工程几十年来一直处理的问题。

1.2 核心观点

本章的核心观点是：AI Agent 并没有让软件工程过时，反而让很多经典软件工程思想重新变得重要。LLM 引入了一种新的高能力计算节点，但围绕这个节点构建可靠、可扩展、低成本系统，仍然需要分层、抽象、缓存、索引、调度、无状态、计算与存储分离等工程原则。

从这个视角看，Agent 不是单纯的“更聪明的聊天框”。它更像一个应用层运行时，负责把用户意图、长期记忆、项目文件、工具调用和模型推理组织成一个完整的执行过程。

1.3 传统计算机工程中的对应概念

经典计算机系统很少把所有能力都放在一个组件里。Web 服务通常把计算逻辑、缓存、数据库、消息队列、对象存储和外部 API 分开；分布式系统强调服务无状态、状态外置、请求可重试、资源可水平扩展；数据库系统强调索引、查询优化、buffer pool 和执行计划。

这些思想背后的共同目标是：不要让昂贵资源做不必要的工作。数据库不希望每次全表扫描；分布式系统不希望每次请求都跨多个服务；操作系统不希望每次访问都落到磁盘。今天的 AI Agent 也遇到了相似问题：不要把所有历史、所有文档、所有工具结果都塞给 LLM；不要把所有任务都交给最贵的大模型；不要让模型重复做已经可以被规则、小模型或缓存完成的事情。

1.4 AI Agent 中的实现形式

当我们把这些思想放到 Agent 里，会看到一组新的名字：Memory、RAG、Context Engineering、Tool Calling、Planner、Router、Distillation、MCP、Multi-Agent。它们听起来是 AI 时代的新概念，但背后的许多系统动机很熟悉。

Memory 像外部状态存储；Context Window 像一次请求的 working set；RAG 和检索像查询相关数据页；AGENTS.md 像一个项目级 Prompt Index；Planner 像调度器；Tool Calling 像 RPC；MCP 像工具协议；Distillation 和小模型像把昂贵计算迁移到更低成本的计算层。名称改变了，但工程问题仍然是资源管理和系统优化。

1.5 不能过度类比的部分

类比有助于理解，但不能代替证明。AI Agent 和传统系统之间也有重要差异。传统 API 多数是确定性的，而 LLM 出具有概率性；存命中后返回固定，而小模型会生成果并可能犯；数据有明确 schema，而自然语言任务常常边界模糊。

所以本文不会说“Agent 就是数据库”或“LLM 就是 CPU”。更准确的说法是：LLM 成为新的计算对象之后，计算机工程里很多成熟的设计原则被重新应用到了 AI 系统中。类比的值在于帮助我们发现结构相似的问题，而不是把两个系统强行等同。

1.6 工程分析

从工程角度看，AI Agent 的关键挑战不是让模型单次回答更好，而是让整个系统持续、稳定、低成本地完成任。这意味着 Agent 需要做三类事情。第一类是上下文管理：决定哪些信息应该进入模型；第二类是资源调度：决定使用规则、小模型还是大模型；第三类是状态管理：决定哪些结果写回 Memory、文件系统或外部工具。

这三类事情本质上都是系统设计问题。它们决定了 Agent 能否从一次性的对话工具，升级成可以长期工作的数字系统。

1.7 本章小结

本章建立了全文的基本视角：AI Agent 的快速发展不是单纯依赖模型变强，而是模型能力与经典计算机工程思想结合的结果。LLM 是新的高能力计算节点，但 Agent 系统真正要解决的是如何围绕这个节点管理上下文、状态、工具和成本。

1.8 Agent 与经典计算机工程的快速对应

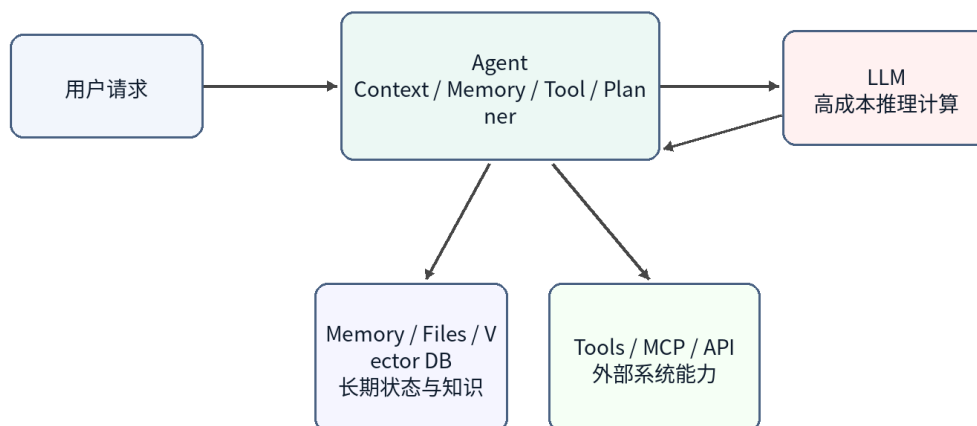
AI Agent 概念	典工程概念	说明
LLM	Compute Engine	负责高能力推理计算，但不应保存全部状态
Memory	Database / Cache	保存长期偏好、项目背景、任务历史
Context Window	Working Set / Buffer Pool	当前求真正加到算的数据
AGENTS.md	Index / Router	用小入口定位需要加载的项目知识
Tool Calling	RPC / API	调用外部软件系统完成真实操作
Planner	Scheduler / Workflow Engine	拆解任务并决定执行顺序
Small Model	Tiered Compute	低成本处理常见任务，复杂任务再升级

注意：这些对应关系用于帮助思考，不代表二者完全等同。

第 2 章 LLM：一种新的 Compute Engine

本章定位：把 LLM 放回计算节点的位置，而不是把它当成整个系统。

LLM 作为一种 Compute Engine



图示为抽象架构，不代表某个产品的官方实现。

图 2

图 2：LLM 作为 Agent 系统中的高成本计算引擎

2.1 本章问题

很多人在讨论 AI 系统时会把 ChatGPT、Claude、Gemini 这样的产品和底层 LLM 混在一起。实际上，产品不是模型本身。产品通常包含前端、Agent 层、Memory、工具调用、权限系统、文件系统、监控、计费和安全策略。LLM 只是其中最重要、也最昂贵的计算节点。

把 LLM 看成 Compute Engine，有助于我们重新理解 Agent 架构：模型不负责保存你的长期记忆，也不天然知道你的项目历史；它只是根据本次请求里的上下文进行推理。真正负责恢复状态、选择信息、调用工具的是 Agent 层。

2.2 LLM 的系统定位

在传统系统里，一个昂贵的计算服务通常不会直接暴露给所有业务逻辑随意调用，而是会被一层服务封装。比如搜索引擎背后有索引服务，数据库背后有查询优化器，GPU 推理服务前面可能有批处理、缓存和路由。

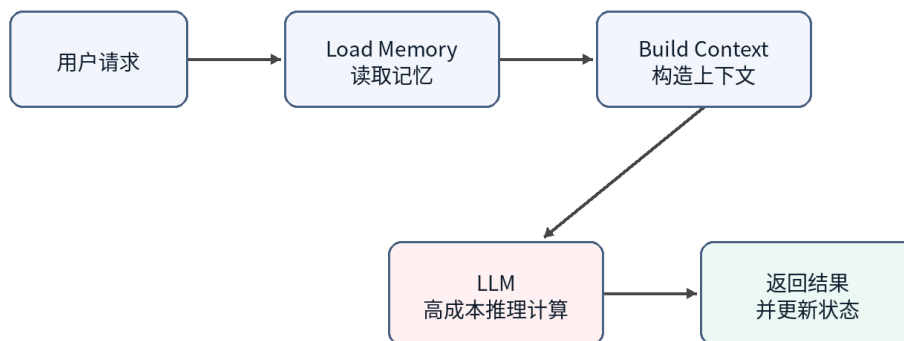
LLM 也应该这样理解。它可以处理自然语言、代码、推理和规划，但这并不意味着所有状态都应该放进模型，也不意味着所有任务都应该直接调用最大模型。LLM 更像一个非常强但成本很高的远程计算服务。每次调用都会产生输入 token、输出 token、延迟和 GPU 资源消耗。

2.3 Stateless Compute : 模型本身不保存会话状态

从一次推理请求的角度看，LLM 基本可以被视为 stateless compute。它不会因为上一轮和某个用户聊过，就在模型参数里永久记住这个用户。下一次推理时，如果上下文里没有提供相关信息，模型就无法可靠地知道之前发生过什么。

因此，所谓“模型记住了我”，在多数产品里并不是模型权重发生了个人化更新，而是 Agent 层把 Memory、近期对话、文件片段、工具结果重新拼接进 Prompt。模型看起来像记得，是因为每次请求前都被重新提供了必要状态。

Agent 的请求生命周期



图示为抽象架构，不代表某个产品的官方实现。

图 3

图 3：Stateless LLM 需要 Agent 在每次请求前恢复上下文

2.4 LLM API 是一种昂贵的远程调用

从工程成本看，调用 LLM API 很像调用一个昂贵的远程服务。它比普通函数调用慢，比大多数数据库查询贵，而且输出还不是完全确定的。因此 Agent 设计中一个非常核心的目标就是减少不必要的大模型调用。

这与传统系统优化非常类似。以前我们会减少跨服务 RPC、减少数据库查询、减少磁盘 IO；现在我们还要减少无意义的 LLM 调用、减少冗余上下文、减少重复推理。Token、上下文长度、模型层级和调用次数，正在成为 AI 系统新的性能指标。

2.5 Token 与 Compute Cost 不是同一个问题

这里需要区分 token 数量和计算成本。这个区分很容易被讲错。比如有人会说“通过模型蒸馏减少 token 使用”，但严格说，蒸馏主要减少的不是 token 数量，而是处理同样 token 所需要的算力成本。一个小模型和一个大模型可能吃进去同样多的 token，但小模型的单 token 成本更低。

可以把 Agent 的模型调用成本拆成一个简单公式：

总成本 $\approx$ 调用次数 $\times$ 单次 token 量 $\times$ 单 token 成本

这三个因子对应三组不同的优化。Planner、缓存命中和任务合并主要减少调用次数；Context Engineering、RAG、裁剪、摘要、Prompt Caching 和工具输出压缩主要减少单次 token 量；蒸馏、小模型、路由和级联主要减少单 token 成本。

维度	降 Token 量	降单 Token 算力成本
优化对象	每次求送和吐出的 token 数	处理同样 token 所耗的算力和价
分布式类比	减少 RPC、缩小 payload、减少往返	把任务下沉到更便宜的计算层，例如冷热分层或服务降级
典型手段	Context Engineering、RAG、裁剪、摘要、Prompt Caching、压缩工具输出、Planner 迭代上限	蒸、小模型、路由、，先用便宜模型尝试，低置信再升级
是否需要和数据	通常不需要，工程和配置即可	蒸馏需要标注、训练、评估和部署；路由或现成小模型不一定需要
落地成本和见效速度	低成本、见效快，改 prompt、检索和缓存策略就可能有效	蒸馏成本高；路由中等；收益依赖任务稳定性和评估体系
主要	裁剪减少上下文表，模型基于不完整信息回答	路由错档，简单任务上大模型会浪费，复杂任务下放小模型会出错
不确定性来源	检索召回质量、摘要保真度、工具输出压缩是否损坏信息	小模型泛化边界、置信度估计是否可靠
独立开发者优先级	先做，门槛低，几乎不需要管	稳定流量和数据积累之后再考虑，蒸馏不是起手式

因此更准确的说法是：蒸馏主要减少对昂贵大模型的使用，而不是必然减少 token。只有当蒸馏让工作流从多次大模型调用变成一次小模型处理，或者减少了多轮规划过程时，token 总量才可能随之下降。此时下降的是被省掉的调用和中间上下文，而不是“蒸馏”直接压缩了每次请求里的 token。

这个区分对真实产品很重要。对独立开发者来说，蒸馏是重武器：需要标注数据、训练管线、评估集、部署和模型托管。相比之下，减少 token 的那组手段更像常规系统优化：缓存稳定前缀、裁剪无关上下文、压缩工具输出、限制 Planner 迭代次数、用路由避免无意义的大模型调用。这些通常不需要训练，改工程结构就能见效。

所以如果目标是先把成本降下来，更现实的顺序通常是：先榨干调用次数和 token 量，再考虑路由、小模型和蒸馏。蒸馏适合放在最后，尤其适合那些已经有稳定流量、稳定任务分布和足够样本的工作流。

2.6 与传统 Compute 的类比

LLM 作为 Compute Engine，可以类比数据库里的执行引擎、云上的远程计算服务、或者 GPU 推理集群。它的能力强，但不应该承担系统中的所有职责。就像数据库不会负责业务流程，CPU 不会负责操作系统调度，LLM 也不应该被看成整个 Agent 系统本身。

这个视角让我们更容易理解为什么 Agent 层重要。Agent 的任务不是取代模型，而是让模型在正确的时间、拿到正确的上下文、以合理的成本完成计算。

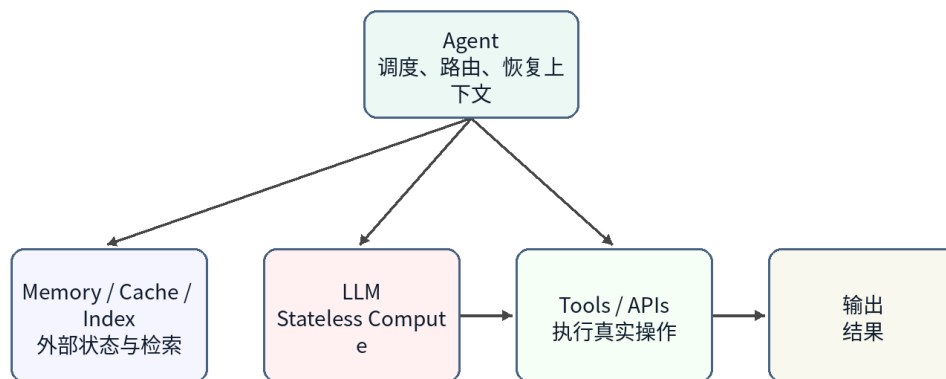
2.7 本章小结

本章把 LLM 定位为新的高能力 Compute Engine。它强大、昂贵、基本无状态，并且通过 API 被 Agent 调用。这个定位解释了为什么 Memory、Context、Tool、Planner 等能力不应该被简单归入模型本身，而应该被视为 Agent 系统的组成部分。

第 3 章 Agent：为什么它更像 Orchestrator

本章定位：Agent 是调度和编排层，负责让合适的资源处理合适的任务。

Agent 作为 Orchestrator



图示为抽象架构，不代表某个产品的官方实现。

图 4

图 4：Agent 更像 Orchestrator，而不只是聊天机器人

3.1 本章问题

如果 LLM 是 Compute Engine，那么 Agent 是什么？一个常见误解是把 Agent 理解成“大模型的外壳”或者“带工具的聊天机器人”。这个说法抓住了一部分现象，但没有抓住架构本质。

更准确地说，Agent 是一个 Orchestrator。它负责接收用户目标，恢复相关状态，选择需要的上下文，决定是否调用工具，判断使用哪个模型，并把多步过程组织成一个可执行的任务流。

3.2 Agent 的核心组成

一个典型 Agent 至少包含几个部分。Memory 保存长期偏好、项目背景和任务历史；Context Builder 决定哪些信息进入当前 Prompt；Planner 把用户目标拆成步骤；Tool Layer 负责调用外部系统；Router 或 Scheduler 决定使用规则、小模型还是大模型；Evaluator 或 Guardrail 用来判断结果是否足够可靠。

这些组件可以在物理实现上分布在多个服务中，也可以被封装在一个产品内部。但从逻辑架构看，它们共同构成 Agent，而不是 LLM 的一部分。

3.3 Agent 的典型执行流程

一个 Agent 接到请求后，通常不会立刻把用户原文扔给大模型。更合理的流程是：先识别任务类型，再读取相关 Memory 或文件，再构造上下文，然后选择计算资源。如果是简单规则可以处理

的任务，就不用模型；如果是常见模式，可以交给小模型；如果复杂度高或风险高，再升级到大模型。

这就是 Agent 与普通聊天机器人的差别。聊天机器人偏向“输入一句话，输出一句话”；Agent 偏向“接收一个目标，组织一组计算与操作来完成它”。

3.4 Agent 与 API Gateway / Scheduler 的关系

从系统类比看，Agent 有点像 API Gateway、Workflow Engine 和 Scheduler 的混合体。它对外暴露统一入口，对内连接多个资源：模型、工具、Memory、文件、外部服务。它还需要根据任务复杂度和成本选择执行路径。

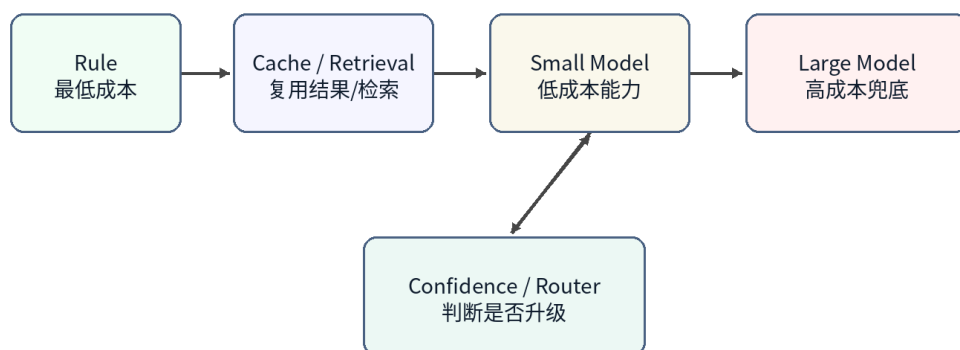
这正是 Orchestrator 的职责：它不一定自己完成所有计算，但它决定哪些资源参与、以什么顺序参与、结果如何合并，以及失败时是否重试、降级或升级。

3.5 分层计算：Rule、Cache、Small Model、Large Model

Agent 的资源调度可以进一步抽象为分层计算。最低层是规则，成本最低、速度最快，但覆盖范围有限。下一层是缓存或检索，可以复用已有结果或找到相关上下文。再往上是小模型，它不像缓存那样保存固定答案，而是保存通过蒸馏获得的能力。最高层是大模型，能力最强、成本最高，适合处理复杂和未知问题。

这个结构和传统系统中的多级缓存、冷热分层、服务降级非常相似。真正优秀的 Agent 不应该让每个请求都直接落到最昂贵的大模型上，而应该尽量在便宜层解决问题。

分层计算：从便宜资源到昂贵模型



图示为抽象架构，不代表某个产品的官方实现。

图 5

图 5：分层计算让 Agent 避免所有请求都落到大模型

3.6 小模型为什么不是普通 Cache

小模型经常被类比成缓存，但这个类比需要修正。普通缓存保存的是结果，命中后直接返回；小模型保存的是能力，它会根据输入重新推理。它不能保证像 Redis 一样返回固定值，但它可以泛化到训练集中没有出现过的新问题。

所以更准确的说法是：蒸馏后的小模型像一种 capability cache，或者说是把大模型在大量样本上表现出的能力压缩成较低成本的计算层。它不是消灭计算，而是把计算从昂贵模型迁移到便宜模型。

3.7 Agent 的局限与风险

Agent 作为 Orchestrator 也带来新的风险。第一，路由错误会导致简单任务被过度调用大模型，或者复杂任务被错误交给小模型。第二，Memory 索引模型基于上下文回答。第三，多步工具调用会放大局部错误。第四，概率模型的不确定性使传统测试方法不完全适用。

因此 Agent 系统需要可观测性、日志、审计、回放、评估集和升级策略。这些再次说明 Agent 已经不是一个单纯的对话界面，而是一个需要系统工程方法来设计和运维的软件系统。

3.8 本章小结

本章将 Agent 定位为 Orchestrator。它不是模型本身，而是围绕模型组织 Memory、Tool、Planner、Context 和资源调度的系统层。这个定位为后续章节铺垫了基础：Memory 为什么像 Storage，AGENTS.md 为什么像 Index，Distillation 为什么像 Tiered Compute，以及 Multi-Agent 为什么会越来越接近分布式系统。

后续章节写作计划

第一部分完成了前三章的基础架构视角。后续章节将沿着这条主线继续展开：Memory 如何对应 Storage，Stateless Agent 如何对应无状态服务，Context Engineering 为什么像 Query Optimization，AGENTS.md 为什么像 Index，以及 Distillation 和 Small Model 为什么更适合放在 Tiered Compute 的框架下理解。

后续写作还需要补上三块更能体现工程差异化的内容。第一，Agent 安全不能只作为风险提示处理。OS 的安全模型会迁移到 Agent 系统里：Prompt Injection 更接近 exploit，工具调用需要权限边界，代码执行和外部系统访问需要 Sandbox。第二，并发调度是 OS 类比最直接的一部分，多 Agent、多租户、多任务队列和资源隔离，比单任务 Planner 更接近 Scheduler。第三，Agent 应该被当成生产系统来讨论：它需要 SLA、幂等、乐观锁、状态机、Retry、降级、升级、可观测、审计、回放和对账。这是本文区别于只讨论“能跑起来”的 OS-analogy 架构文章的重点。

1. 第 4 章将拆解 Memory、Tool、Planner 的边界。
1. 第 5-6 章将重点讨论 Compute / Storage Separation 与 Stateless。
1. 第 7-8 章将把 Context Engineering、AGENTS.md 与数据库优化建立联系。
1. 第 9-10 章将讨论 Retrieval、Token Reduction、Distillation、Small Model 和分层计算，重点拆开“减少 token 量”和“降低单 token 算力成本”这两条不同优化轴。
1. 第 11 章将把 Agent 当成生产系统来讨论，重点包括幂等、乐观锁、状态机、retry、降级、升级、可观测、审计、回放和对账。
1. 第 12 章将讨论 Multi-Agent、多租户和并发调度，把 Planner 与 Scheduler 的类比从单任务执行推进到资源竞争和任务编排。
1. 第 13 章将补上 Agent 安全模型，重点包括 Prompt Injection 作为 exploit、工具权限边界、Sandbox、最小权限和审计。
1. 第 14 章将回到 Agent Operating System，把 Memory、Tool、Planner、安全、调度和生产可靠性放回统一系统视角。